May I ask how are you processing the files when I upload it? Is there any difference between smaller and bigger file handling?
put a small sequence detailing how you are processing files when uploaded. whats happening in client, server, OCR, LLM, etc.., when a file is uploaded. I have an requirement in my application where i need to read and understand what is inside the file. i might get docx, pdf (with or without images), images (PNG/JPEG). I have paddleOCR, GPT LLM at my disposal (They are hosted out side of my application as 2 different endpoints.). Requirement is to process the files as fast as possible like you do. hence requested this info from your side.
i need to process this in java platform. And the file processing output should be always markdown. suggest your answer based on that.

1) provide info on how you are doing the processing
2) provide info on how i should do the processing

File processing pipelines split workflows based on whether text is natively selectable or locked inside images, routing documents through specialized extraction layers before passing structured text to an LLM for final Markdown synthesis.

① Client Upload

Asynchronous multipart transmission

The client uploads the file (PDF, DOCX, or image) via a multipart HTTP POST request. Large files utilize chunked uploads or stream directly to storage to prevent memory bottlenecks.

② Server Routing & Preprocessing

Type-based text extraction vs. image rendering

The server inspects the MIME type. Native text documents (DOCX, text-based PDFs) are parsed directly to extract text and layout metadata. Scanned PDFs and image files (PNG/JPEG) have their pages rendered into high-resolution images.

③ **OCR Integration Layer**

Concurrent PaddleOCR endpoint calls

For images or scanned pages, the server dispatches concurrent HTTP requests to your external PaddleOCR endpoint. PaddleOCR returns recognized text blocks, confidence scores, and bounding box coordinates to preserve reading order.

④ **LLM Markdown Synthesis**

GPT endpoint formatting

The server aggregates raw text, OCR results, and structural metadata into an optimized prompt, sending it to the GPT LLM endpoint with strict instructions to output clean, valid GitHub-Flavored Markdown.

⑤ **Delivery**

Response transmission

The resulting Markdown string is returned to the client (or streamed if using server-sent events for real-time rendering).

Handling Small vs. Big Files

- **Small Files (Single images, short PDFs, short DOCX):** Handled synchronously in a single request cycle. Text extraction and OCR occur immediately, and the payload fits comfortably within a single GPT context window for instant generation.
- **Big Files (Multi-page PDFs, heavy scans):** Require asynchronous processing queues, parallel page splitting (using multithreading), batching requests to PaddleOCR, and handling LLM token constraints through chunked processing or structural mapping.

Implementing Fast File Processing in Java

To achieve maximum speed matching high-performance AI services, your Java backend should leverage non-blocking operations, parallel execution, and efficient local parsers before hitting your external endpoints.

Recommended Java Tooling

- **Document Parsers:** Use **Apache POI** (`XWPFDocument`) for `.docx` files and **Apache PDFBox** for native PDF text extraction and converting PDF pages into `BufferedImage` objects.

- **Concurrency & Speed:** Use Java's `CompletableFuture` or reactive execution (like Spring WebFlux) to process pages concurrently. For instance, render a 20-page PDF into images and fire asynchronous requests to PaddleOCR for all pages simultaneously rather than sequentially.
- **HTTP Communication:** Use Java's native `HttpClient` (Java 11+) with HTTP/2 enabled for high-throughput, low-latency calls to your external PaddleOCR and GPT endpoints.

Optimizing for Markdown Output

- **Layout Preservation:** Instruct your GPT endpoint explicitly via system prompts to map headings to `#`, bullet points to `-`, and tabular data to Markdown tables (`| Col 1 | Col 2 |`).
- **OCR Sorting:** When processing OCR results from PaddleOCR, sort text bounding blocks top-to-bottom and left-to-right before sending them to the LLM so the reading order remains intact.

1) How do you think i should process the files if the requirement is to support max file size of 20MB and the processing time should be less than 30 seconds?
2) Give a small sequence explaining how the flow should b

To meet a strict 30-second processing SLA for payloads up to 20MB, you must avoid disk I/O entirely by processing byte streams in memory and fanning out network calls. Leveraging a Spring Boot environment with an asynchronous executor pool allows you to extract pages and execute OCR tasks in parallel rather than sequentially.

① In-Memory Ingestion

Memory-only processing

The application receives the `MultipartFile` and reads it directly into RAM as an `InputStream`. No temporary files are written to the server's disk, eliminating I/O bottlenecks.

② Format Detection & Extraction

PDFBox & POI Execution

The engine routes the stream. DOCX files are parsed instantly. PDFs are checked for native text layers; if none exist (scanned document), the engine rasterizes the PDF pages into `BufferedImage` objects at 300 DPI.

③ Concurrent OCR Fan-Out

Parallel API calls

For images or rendered PDF pages, the application spins up multiple `CompletableFuture` tasks. All pages are sent to the external PaddleOCR endpoint simultaneously over HTTP/2 connections.

④ Aggregation & LLM Formatting

Structured context assembly

The application waits for all futures to resolve, concatenates the text in the correct page order, and sends a single, comprehensive prompt to the GPT LLM to format the payload into clean Markdown.

Document Processing Implementation

This implementation utilizes `Apache POI` for DOCX, `Apache PDFBox` for PDFs, and Java's `CompletableFuture` to run the PaddleOCR network calls in parallel, ensuring the 30-second threshold is met even for multi-page image PDFs.

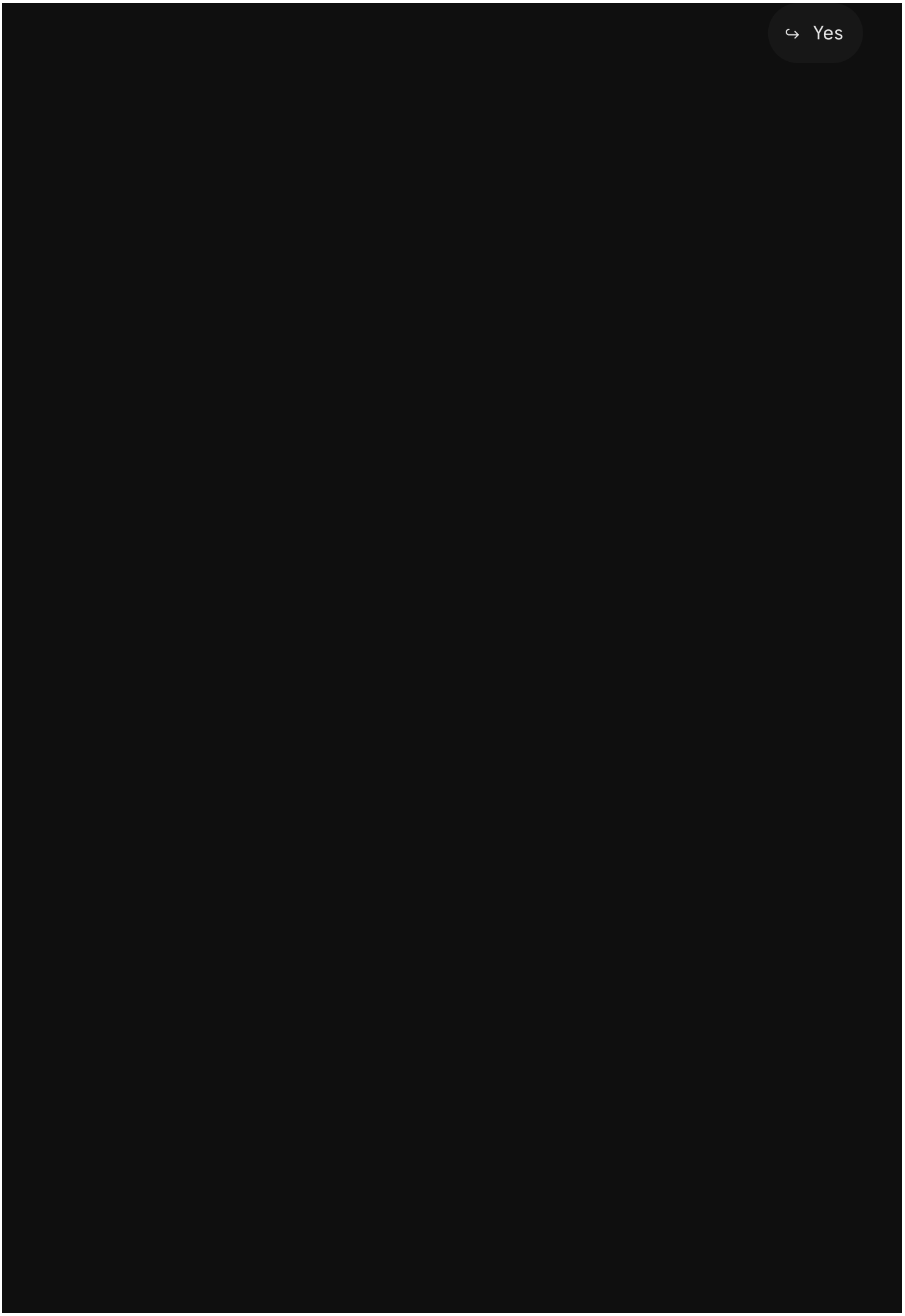
Java

```java
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.rendering.PDFRenderer;
import org.apache.pdfbox.text.PDFTextStripper;
import org.apache.poi.xwpf.extractor.XWPFWordExtractor;
import org.apache.poi.xwpf.usermodel.XWPFDocument;
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;

import javax.imageio.ImageIO;
import java.awt.image.BufferedImage;
import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.InputStream;
```

```java
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.stream.Collectors;

@Service
public class DocumentProcessingService {

    // Thread pool dedicated to parallel OCR network calls
    private final ExecutorService ocrExecutor = Executors.newFixedThreadPool(10);

    public String processFile(MultipartFile file) throws Exception {
        String contentType = file.getContentType();
        String extractedText = "";

        if (contentType == null) {
            throw new IllegalArgumentException("Unknown file type");
        }

        try (InputStream inputStream = file.getInputStream()) {
            if (contentType.contains("wordprocessingml.document") || file.getOriginal
                extractedText = processDocx(inputStream);
            } else if (contentType.contains("pdf") || file.getOriginalFilename().ends
                extractedText = processPdf(inputStream);
            } else if (contentType.startsWith("image/")) {
                BufferedImage image = ImageIO.read(inputStream);
                extractedText = callPaddleOcr(image);
            } else {
                throw new IllegalArgumentException("Unsupported file type: " + conten
            }
        }

        // Send the aggregated raw text to GPT for Markdown structuring
        return callGptLlmForMarkdown(extractedText);
    }

    private String processDocx(InputStream inputStream) throws IOException {
        try (XWPFDocument doc = new XWPFDocument(inputStream);
             XWPFWordExtractor extractor = new XWPFWordExtractor(doc)) {
            return extractor.getText();
        }
    }

    private String processPdf(InputStream inputStream) throws Exception {
        try (PDDocument document = PDDocument.load(inputStream)) {
            PDFTextStripper stripper = new PDFTextStripper();
            String nativeText = stripper.getText(document);
```

```
            // If PDF has native text, return it immediately (fastest path)
            if (nativeText != null && !nativeText.trim().isEmpty()) {
                return nativeText;
            }

            // Fallback: Scanned PDF. Render pages and OCR concurrently.
            PDFRenderer pdfRenderer = new PDFRenderer(document);
            List<CompletableFuture<String>> ocrFutures = new ArrayList<>();

            for (int page = 0; page < document.getNumberOfPages(); page++) {
                // PDFBox rendering is best done synchronously to avoid thread-safety
                BufferedImage image = pdfRenderer.renderImageWithDPI(page, 300);

                // Fan-out the network calls to PaddleOCR
                CompletableFuture<String> future = CompletableFuture.supplyAsync(() -
                    return callPaddleOcr(image);
                }, ocrExecutor);

                ocrFutures.add(future);
            }

            // Wait for all OCR calls to finish and preserve page order
            return ocrFutures.stream()
                    .map(CompletableFuture::join)
                    .collect(Collectors.joining("\n\n--- Page Break ---\n\n"));
        }
    }

    private String callPaddleOcr(BufferedImage image) {
        // TO BE IMPLEMENTED:
        // 1. Convert BufferedImage to byte array (e.g., JPEG/PNG format)
        // 2. Execute HTTP POST request to external PaddleOCR endpoint
        // 3. Parse JSON response and return extracted String
        return "OCR_EXTRACTED_TEXT_PLACEHOLDER";
    }

    private String callGptLlmForMarkdown(String rawText) {
        // TO BE IMPLEMENTED:
        // 1. Wrap rawText in a system prompt instructing GPT to output strict GitHub
        // 2. Execute HTTP POST to external GPT endpoint
        // 3. Return the formatted Markdown payload
        return "GPT_MARKDOWN_PLACEHOLDER";
    }
}
```

Need help configuring the HTTP client?


↳ Yes